

Validity for Sigma Types: Proof Structure

1 Overview

We describe the extension of the logical relation (validity) for dependent type theory with $\mathsf{U} : \mathsf{U}$ to handle Σ -types. The presentation follows Coquand–Huber (2018), where the logical relation is indexed by *codes* from a domain-theoretic model. The key new difficulty compared to Π -types is in the head-reduction transport lemma.

2 Codes and projections

The set `Code` of codes includes:

$$u, v ::= \perp \mid \mathsf{U} \mid \mathsf{Prop} \mid \lambda g \mid \Pi b f \mid \Sigma b f \mid (u, v)$$

A code $\Sigma b f$ represents a Σ -type whose domain has code b and whose codomain is the finite function f . A code (u, v) represents a pair value.

We use a uniform projection notation for both terms and codes:

$$M.1, \quad M.2 \quad (\text{first and second projection of a term})$$

$$w.1, \quad w.2 \quad (\text{first and second projection of a code})$$

On codes, the projections are defined by:

$$(u, v).1 = u, \quad w.1 = \perp \text{ otherwise}$$

$$(u, v).2 = v, \quad w.2 = \perp \text{ otherwise}$$

3 Validity at Σ -types

Definition 1 (Term validity at Σ). *We define $\Gamma \vdash M : T \mid w : \Sigma b f$ to hold when:*

1. $\text{ValTy } \Gamma T (\Sigma b f)$ (type validity of T), which includes $T \rightarrow \Sigma A B$ with $\Gamma \vdash A : \mathbb{U}$ and $\Gamma, x:A \vdash B : \mathbb{U}$, together with the edge functions;
2. $\Gamma \vdash M.1 : A \mid w.1 : b$ (validity of the first projection);
3. $\Gamma \vdash M.2 : B[M.1] \mid w.2 : f(w.1)$ (validity of the second projection).

Remark 1. When w is not a pair (u, v) , we have $w.1 = \perp$ and $w.2 = \perp$. Since Val at code $\perp$ is trivially $\top$, conditions (2) and (3) are automatically satisfied. This means there is no need for separate case analysis on whether w is a pair or not: the definition is uniform in w .

Remark 2. Compared to the Π -case, where term validity at $\Pi b f$ is defined for $w = \lambda g$ and stores $\text{Val } \Gamma (\text{App } M N) (B[N]) v (f(u))$, the type $B[N]$ does not mention M . For Σ , the second component's type $B[M.1]$ does mention M . This is the source of the difficulty in head-reduction transport.

4 Type validity at Σ

Definition 2 (Type validity at Σ). $\text{ValTy } \Gamma T (\Sigma b f)$ holds when:

- $T \rightarrow \Sigma A B$ and $\Gamma \vdash T \equiv \Sigma A B : \mathbb{U}$;
- $\Gamma \vdash A : \mathbb{U}$ and $\Gamma, x:A \vdash B : \mathbb{U}$;
- $\text{ValTy } \Gamma A b$ (domain type validity);
- For each $(u, v) \in f$ and N with $\Gamma \vdash N : A$ and $\text{Val } \Gamma N A u b$, we have $\text{ValTy } \Gamma B[N] v$ (edge function for values);
- Similarly for the equality edge.

5 Type-equality transport

Lemma 1 (Transport along type equality). For all codes u and a :

$$\text{EqValTy } \Gamma C C' a \implies \text{Val } \Gamma N C u a \implies \text{Val } \Gamma N C' u a$$

Proof. By induction on a . The type parameter C appears in $\text{Val } \Gamma N C u a$ only in two cases: $a = \Pi b f$ with $u = \lambda g$, and $a = \Sigma b f$ with $u = (u', v')$. In these cases the validity record stores a head reduction $C \rightarrow \Pi A B$ or $C \rightarrow \Sigma A B$, and $\text{EqValTy } \Gamma C C' a$ provides the corresponding data for C' , allowing the record to be rebuilt. In all other cases—including $a = \mathbb{U}$, where Val depends on N but not on C —the result is immediate. $\square$

Remark 3. *This lemma plays a central role in the Σ case of head-reduction transport (Lemma 2, Step 4), where it transports $\mathbf{Val}$ from type $B[M'.1]$ to type $B[M.1]$. For Π -types alone, this lemma is not needed in head-reduction transport.*

6 Head-reduction transport

The central lemma for adequacy is:

Lemma 2 (Head-reduction produces equality). *Given:*

- $M \rightarrowtail M'$ and $\Gamma \vdash M \equiv M' : T$,
- $\Gamma \vdash M' : T \mid u : a$ (validity of the reduct),

we can conclude:

$$\Gamma \vdash M' = M : T \mid u : a$$

which bundles $\mathbf{Val} \Gamma M T u a$ and $\mathbf{Val} \Gamma M' T u a$ and the equality $\mathbf{EqVal} \Gamma M' M T u a$.

Remark 4. *The return type is $\mathbf{EqVal}$ (not just $\mathbf{Val}$). This is essential for the Σ case: the equality at the first projection is needed to transport the second projection's type.*

6.1 The Π case

For $a = \Pi b f$ and $u = \lambda g$, the validity data for M' stores, for each $(u_0, v_0) \in g$ and argument N :

$$\mathbf{Val} \Gamma (\mathbf{App} M' N) (B_0[N]) v_0 (f(u_0))$$

Since $M \rightarrowtail M'$, we have $\mathbf{App} M N \rightarrowtail \mathbf{App} M' N$. The type $B_0[N]$ does not mention M , so the induction hypothesis applies directly at code $(v_0, f(u_0))$:

$$\mathbf{EqVal} \Gamma (\mathbf{App} M' N) (\mathbf{App} M N) (B_0[N]) v_0 (f(u_0))$$

No type transport is needed.

6.2 The Σ case

For $a = \Sigma b f$, we work with the uniform definition from Section 3. The validity data for M' includes:

$$\text{Val } \Gamma \ M'.1 \ A \ w.1 \ b \quad (1)$$

$$\text{Val } \Gamma \ M'.2 \ B[M'.1] \ w.2 \ f(w.1) \quad (2)$$

We need to produce the same data for M , plus equality.

Step 1: First projection. From $M \twoheadrightarrow M'$ we get $M.1 \twoheadrightarrow M'.1$. From $\Gamma \vdash M \equiv M' : \Sigma A B$ we derive (by the congruence rule for projections):

$$\Gamma \vdash M.1 \equiv M'.1 : A$$

Together with (1), the induction hypothesis at code $(w.1, b)$ gives:

$$\text{EqVal } \Gamma \ M'.1 \ M.1 \ A \ w.1 \ b \quad (3)$$

This bundles $\text{Val } \Gamma \ M.1 \ A \ w.1 \ b$.

Step 2: Type transport via the edge function. The equality (3) and the equality edge function from ValTy at $\Sigma b f$ give:

$$\text{EqValTy } \Gamma \ B[M'.1] \ B[M.1] \ f(w.1) \quad (4)$$

This is the key step: the *equality* edge function converts the equality of *values* ($M'.1 = M.1$) into an equality of *types* ($B[M'.1] = B[M.1]$). This step has no analogue in the Π case.

Note that the *value* edge (which only takes Val data) would not suffice here: we genuinely need the *equality* at the first projection to produce the type equality for the transport. This is why Lemma 2 must return EqVal rather than just Val .

Step 3: Second projection. From $M \twoheadrightarrow M'$ we get $M.2 \twoheadrightarrow M'.2$. The congruence rule for the second projection, applied to $\Gamma \vdash M' \equiv M : \Sigma A B$ (note the swap, so that $M'.1$ appears in the type), gives:

$$\Gamma \vdash M'.2 \equiv M.2 : B[M'.1]$$

Together with (2), the induction hypothesis at code $(w.2, f(w.1))$ gives:

$$\text{EqVal } \Gamma \ M'.2 \ M.2 \ B[M'.1] \ w.2 \ f(w.1) \quad (5)$$

In particular, $\text{Val } \Gamma \ M.2 \ B[M'.1] \ w.2 \ f(w.1)$.

Step 4: Transport. The validity data for M at Σ requires:

$$\text{Val } \Gamma \ M.2 \ B[M.1] \ w.2 \ f(w.1)$$

We have **Val** at $B[M'.1]$ from Step 3, and **EqValTy** between $B[M'.1]$ and $B[M.1]$ from Step 2. Lemma 1 gives $\text{Val } \Gamma \ M.2 \ B[M.1] \ w.2 \ f(w.1)$. $\square$

7 Modular structure

The proof has the following modular structure:

1. **Domain operations** ($w.1$, $w.2$, **Eval**): Pure operations on codes, no typing.
2. **Validity definition**: Uniform in w using $w.1$ and $w.2$. No case split on whether w is a pair.
3. **Transport lemmas** (sym, trans, fwd, restrict, up, down, Sup): Each proved by induction on codes. The Σ cases follow the same pattern as Π because the definition is uniform.
4. **Head-reduction transport** (Lemma 2): Returns **EqVal** (not just **Val**). The 4-step chain in Section 6.2 uses:
 - the induction hypothesis at the first projection (Step 1),
 - the edge function to get type equality (Step 2),
 - the induction hypothesis at the second projection (Step 3),
 - the standard **EqValTy**-transport (Step 4).
5. **Adequacy**: Uses head-reduction transport for the β -reduction cases (application, $.1\text{-}\beta$, $.2\text{-}\beta$).

8 Comparison with Π

	Π	Σ
Value code	λg	any w (uniform via $w.1/w.2$)
Stored data	$\text{Val } (\text{App } M \ N) \ B[N] \ \dots$	$\text{Val } M.1 \ A \ \dots$ and $\text{Val } M.2 \ B[M.1] \ \dots$
Type depends on M ?	No ($B[N]$)	Yes ($B[M.1]$)
HeadRed transport	Direct IH	IH + edge + transport
Returns EqVal ?	Useful	Essential